Geometry-Aware Prototype-Conditioned Learning in Feed Forward Neural Networks

NIVHSA
India

March 27, 2026

Abstract

We propose a geometry-aware learning framework for feed forward neural networks in which local structure in hidden representation space modulates the effective parameters used for prediction. Instead of applying a single globally shared output mapping to every sample, we associate the hidden representation with a small set of latent prototypes, compute soft prototype memberships, and use these memberships to form an input-dependent mixture of output experts. We further introduce an auxiliary local target in hidden space, defined by the same memberships, so that the hidden representation is regularized by local geometry. The resulting model is trained end-to-end by gradient descent on a differentiable objective consisting of the supervised output loss and the auxiliary local loss. The method is motivated by the hypothesis that destructive interference between distant data regions can be reduced when learning is informed by local geometry in representation space.

1 Introduction

In a standard feed forward neural network (FFNN), the same parameter tensors are shared across all samples, and training is typically driven by a global output loss propagated through the entire network. This paradigm has been highly successful, but it also couples the updates induced by all samples through a common parameter set. When two samples arise from distinct regions of the data distribution, their induced updates may interfere.

The central idea of this work is that neural learning should depend not only on the global output error, but also on the *local geometry* of the hidden representation space. We therefore augment a one-hidden-layer FFNN with three ingredients:

1. latent prototypes in hidden space,
2. soft memberships of each sample to those prototypes,
3. and a prototype-conditioned auxiliary target in hidden space.

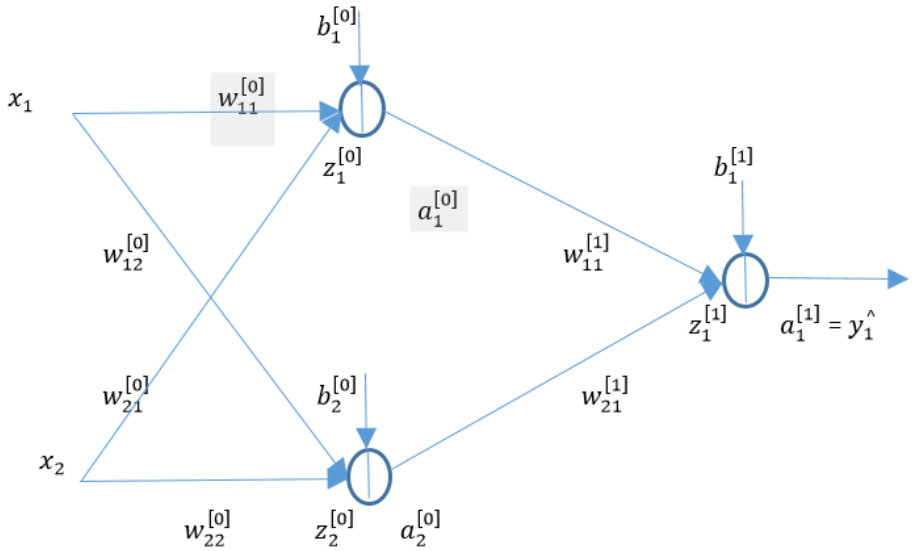

Figure 1: Backbone one-hidden-layer feed forward neural network. In the proposed method, the hidden representation is further coupled to latent prototypes, soft prototype memberships, a prototype-conditioned mixture of output experts, and an auxiliary local loss. The figure should therefore be interpreted as the base architecture on top of which the geometry-aware mechanism is added.

The memberships determine how strongly each sample uses a collection of output experts, thereby producing input-dependent effective parameters. The same memberships also define a local hidden-space target, which regularizes the hidden representation.

For clarity and consistency with the implementation, the model studied in this paper is a one-hidden-layer FFNN trained end-to-end by automatic differentiation. The theoretical analysis later in the paper is presented as a simplified mechanism analysis that isolates the interference-reduction effect of prototype-conditioned gating; it is not claimed to be a full convergence analysis of the exact nonlinear optimization dynamics.

2 Geometry-aware prototype-conditioned model

2.1 Backbone hidden representation

Let $\mathbf{x} \in \mathbb{R}^d$ be the input. We consider a one-hidden-layer FFNN with hidden dimension H. The hidden representation is

$$\mathbf{h}(\mathbf{x}) = \tanh(W_h \mathbf{x} + \mathbf{b}_h), \tag{1}$$

where $W_h \in \mathbb{R}^{H\times d}$ and $\mathbf{b}_h \in \mathbb{R}^H$ are trainable parameters.

For a C-class classification problem, the network ultimately outputs logits in $\mathbb{R}^C$ and probabilities

$$\mathbf{p}(\mathbf{x}) = \mathrm{softmax}(\mathbf{z}(\mathbf{x})). \tag{2}$$

2.2 Latent prototypes and soft memberships

We introduce K trainable latent prototypes

$$\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_K \in \mathbb{R}^H$$

in hidden representation space. For an input $\mathbf{x}$, the hidden representation $\mathbf{h}(\mathbf{x})$ is softly assigned to these prototypes via

$$g_k(\mathbf{x}) = \frac{\exp\left(-\frac{\|\mathbf{h}(\mathbf{x})-\mathbf{c}_k\|_2^2}{\tau}\right)}{\sum_{r=1}^{K}\exp\left(-\frac{\|\mathbf{h}(\mathbf{x})-\mathbf{c}_r\|_2^2}{\tau}\right)}, \qquad k = 1, \ldots, K, \tag{3}$$

where $\tau > 0$ is a temperature parameter.

The membership vector

$$\mathbf{g}(\mathbf{x}) = \big(g_1(\mathbf{x}), \ldots, g_K(\mathbf{x})\big)$$

encodes the local geometry of $\mathbf{x}$ in hidden space.

2.3 Prototype-conditioned output experts

Instead of one global output map, we use K output experts:

$$W_1^{\mathrm{exp}}, \ldots, W_K^{\mathrm{exp}} \in \mathbb{R}^{C\times H}, \qquad \mathbf{b}_1^{\mathrm{exp}}, \ldots, \mathbf{b}_K^{\mathrm{exp}} \in \mathbb{R}^C.$$

Each expert produces logits

$$\mathbf{z}_k^{\mathrm{exp}}(\mathbf{x}) = W_k^{\mathrm{exp}}\,\mathbf{h}(\mathbf{x}) + \mathbf{b}_k^{\mathrm{exp}}. \tag{4}$$

The final logits are a membership-weighted mixture of these experts:

$$\mathbf{z}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\,\mathbf{z}_k^{\mathrm{exp}}(\mathbf{x}). \tag{5}$$

Equation (5) is the implementation-consistent form of an input-dependent effective output layer. Different regions of hidden space therefore induce different effective parameter usage.

2.4 Geometry-aware local target and auxiliary loss

To make learning explicitly sensitive to local geometry in hidden space, we introduce trainable prototype-associated target vectors

$$\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_K \in \mathbb{R}^H.$$

For a given input $\mathbf{x}$, the auxiliary target is defined by

$$\mathbf{t}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, \boldsymbol{\phi}_k. \tag{6}$$

The auxiliary local loss is then

$$\mathcal{L}_{\text{loc}}(\mathbf{x}) = \frac{1}{2} \left\| \mathbf{h}(\mathbf{x}) - \mathbf{t}(\mathbf{x}) \right\|_2^2. \tag{7}$$

Thus, the same local geometry information that determines the effective output map also determines a prototype-conditioned regularization target for the hidden representation.

2.5 Supervised loss and end-to-end objective

For classification, let $\mathbf{y} \in \{0,1\}^C$ denote the one-hot label vector. The supervised output loss is

$$\mathcal{L}_{\text{out}}(\mathbf{x}, \mathbf{y}) = -\sum_{c=1}^{C} y_c \log p_c(\mathbf{x}), \tag{8}$$

where $\mathbf{p}(\mathbf{x})$ is defined in (2).

The implementation-consistent training objective for one sample is

$$\mathcal{L}_{\text{impl}}(\mathbf{x}, \mathbf{y}) = \mathcal{L}_{\text{out}}(\mathbf{x}, \mathbf{y}) + \lambda\, \mathcal{L}_{\text{loc}}(\mathbf{x}), \tag{9}$$

where $\lambda \geq 0$ controls the strength of the geometry-aware local regularization.

Over a training set $\mathcal{D} = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^{N}$, the empirical objective is

$$\mathcal{J}(\Theta) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_{\text{impl}}(\mathbf{x}^{(n)}, \mathbf{y}^{(n)}), \tag{10}$$

where

$$\Theta = \left\{ W_h, \mathbf{b}_h, \{W_k^{\text{exp}}, \mathbf{b}_k^{\text{exp}}\}_{k=1}^{K}, \{\mathbf{c}_k\}_{k=1}^{K}, \{\boldsymbol{\phi}_k\}_{k=1}^{K} \right\}. \tag{11}$$

2.6 Training rule

All trainable quantities in (11) are optimized jointly by gradient descent on the differentiable objective (10). For a minibatch estimate of the loss, the parameter update is

$$\Theta \leftarrow \Theta - \eta\, \nabla_{\Theta} \mathcal{J}(\Theta), \tag{12}$$

where $\eta > 0$ is the learning rate.

This is important: in the implementation used in experiments, the memberships, prototypes, expert parameters, and auxiliary target vectors are all part of a single computational graph. Hence gradients are allowed to flow through the membership function in (3), through the expert mixture in (5), and through the auxiliary target in (6). In this sense, the implemented model is an end-to-end differentiable geometry-aware architecture rather than a manually decoupled layer-wise update scheme.

2.7 Interpretation in terms of local geometry

The role of local geometry enters the model in two coupled ways.

First, the prototype memberships modulate the effective output computation via (5). Samples that lie in different regions of hidden space therefore activate different mixtures of output experts.

Second, the same memberships define the auxiliary target (6), which regularizes the hidden representation toward a geometry-dependent direction. Therefore, the hidden representation is shaped both by the supervised label signal and by the local geometric structure induced by the prototypes.

This is the sense in which the present method uses local geometry information: local distances in hidden representation space influence both the forward computation and the learned hidden-space regularization.

3 Illustration for the one-hidden-layer network

For the one-hidden-layer FFNN in Fig. 1, the hidden representation is

$$\mathbf{h}(\mathbf{x}) = \tanh(W_h \mathbf{x} + \mathbf{b}_h).$$

Given prototypes $\{\mathbf{c}_k\}_{k=1}^{K}$, the memberships are computed by (3). The output logits of the kth expert are

$$\mathbf{z}_k^{\exp}(\mathbf{x}) = W_k^{\exp} \mathbf{h}(\mathbf{x}) + \mathbf{b}_k^{\exp},$$

and the final logits are

$$\mathbf{z}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, \mathbf{z}_k^{\exp}(\mathbf{x}).$$

Thus, the output mapping is no longer globally fixed across all inputs; it depends on the sample's location in hidden representation space. The same memberships define the auxiliary target

$$\mathbf{t}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, \boldsymbol{\phi}_k,$$

which induces the local loss in (7). The model is then trained end-to-end by minimizing (10).

Algorithm 1 Geometry-Aware Prototype-Conditioned End-to-End Training

Require: Training set $\mathcal{D} = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^{N}$, number of prototypes K, learning rate η, temperature τ, local-loss weight λ

```
Initialize hidden-layer parameters W_h, b_h
Initialize prototypes {c_k}_{k=1}^K, local target vectors {φ_k}_{k=1}^K
Initialize expert parameters {W_k^exp, b_k^exp}_{k=1}^K
for each epoch do
    for each minibatch B ⊂ D do
        for each sample (x, y) ∈ B do
            Compute hidden representation h(x) = tanh(W_h x + b_h)
            Compute memberships g_k(x) using (3)
            Compute expert logits z_k^exp(x) using (4)
            Compute mixed logits z(x) = Σ_{k=1}^K g_k(x) z_k^exp(x)
            Compute local target t(x) = Σ_{k=1}^K g_k(x) φ_k
            Compute supervised loss L_out(x, y)
            Compute local loss L_loc(x)
            Form total loss L_impl(x, y) = L_out(x, y) + λ L_loc(x)
        end for
        Compute minibatch average of L_impl
        Backpropagate through the full computational graph
        Update all parameters in Θ using one optimizer step
    end for
end for
```

4 Pseudo-code

5 Discussion

The proposed formulation differs from earlier hand-designed local update proposals in an important way: the present implementation is fully differentiable and internally closed. The prototype memberships, expert parameters, prototypes themselves, and auxiliary target vectors are all learned jointly by gradient descent on the implemented objective. Consequently, there is no ambiguity about how the prototype vectors or local target vectors are updated, and there is no mismatch between the stated objective and the implemented optimizer.

At the same time, the model preserves the central geometry-aware idea: samples in different regions of hidden space induce different effective mixtures of output experts, and the same local geometry defines the auxiliary target used to regularize the hidden representation.

The intended benefit is not necessarily a universal improvement over standard backpropagation, but rather a geometry-aware inductive bias that may reduce destructive interference between distant data regions and improve performance in settings where globally shared updates are suboptimal.

6 A simplified interference-reduction result

The implementation in this paper is trained end-to-end by gradient descent on the differentiable objective (10). The result below should therefore be interpreted as a simplified mechanism analysis that isolates the interference-reduction effect of geometry-aware gating. It is not claimed to be a full convergence theorem for the exact nonlinear optimizer used in the implementation.

6.1 Setup

Consider a one-hidden-layer network with hidden representation $\mathbf{h}(\mathbf{x}) \in \mathbb{R}^d$. For theoretical analysis, suppose the output layer is written as a collection of region-specific matrices

$$W_1, \ldots, W_K \in \mathbb{R}^{m \times d},$$

and define the effective output-layer weight

$$\widetilde{W}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, W_k, \tag{13}$$

where $g_k(\mathbf{x}) \geq 0$ and $\sum_{k=1}^{K} g_k(\mathbf{x}) = 1$.

To isolate the effect of gating, consider the simplified sample-induced update

$$\Delta W_k(\mathbf{x}) = -\eta\, g_k(\mathbf{x})\, \boldsymbol{\delta}(\mathbf{x})\, \mathbf{h}(\mathbf{x})^\top, \tag{14}$$

where $\eta > 0$ is the learning rate and $\boldsymbol{\delta}(\mathbf{x}) \in \mathbb{R}^m$ is a generic error vector. This update is not the exact optimizer of the full implemented model; rather, it captures the geometry-gated rank-one update component that the proposition studies.

To measure interaction between two sample-induced updates, define the total overlap

$$\Omega(\mathbf{x}_p, \mathbf{x}_q) = \sum_{k=1}^{K} \langle \Delta W_k(\mathbf{x}_p),\, \Delta W_k(\mathbf{x}_q) \rangle_F\,, \tag{15}$$

where $\langle A, B \rangle_F = \operatorname{tr}(A^\top B)$ denotes the Frobenius inner product.

6.2 Preliminary lemma

Lemma 6.1. *Let* $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ *and* $\mathbf{r}, \mathbf{s} \in \mathbb{R}^d$. *Then*

$$\left\langle \mathbf{u}\mathbf{r}^\top, \mathbf{v}\mathbf{s}^\top \right\rangle_F = (\mathbf{u}^\top \mathbf{v})(\mathbf{r}^\top \mathbf{s}). \tag{16}$$

Proof. Using the definition of the Frobenius inner product,

$$\left\langle \mathbf{u}\mathbf{r}^\top, \mathbf{v}\mathbf{s}^\top \right\rangle_F = \operatorname{tr}\big((\mathbf{u}\mathbf{r}^\top)^\top (\mathbf{v}\mathbf{s}^\top)\big) = \operatorname{tr}\big(\mathbf{r}\mathbf{u}^\top \mathbf{v}\mathbf{s}^\top\big)\,.$$

Since $\mathbf{u}^\top \mathbf{v}$ is a scalar,

$$\operatorname{tr}\big(\mathbf{r}\mathbf{u}^\top \mathbf{v}\mathbf{s}^\top\big) = (\mathbf{u}^\top \mathbf{v})\operatorname{tr}(\mathbf{r}\mathbf{s}^\top) = (\mathbf{u}^\top \mathbf{v})(\mathbf{r}^\top \mathbf{s}),$$

which proves the claim. □

6.3 Main proposition

Proposition 6.2 (Reduction of cross-region update overlap)**.** *Consider two samples* $\mathbf{x}_p$ *and* $\mathbf{x}_q$ *with updates defined by* (14)*. Then their total overlap satisfies*

$$\Omega(\mathbf{x}_p, \mathbf{x}_q) = \eta^2 \Big(\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)\Big)\Big(\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)\Big) \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q). \tag{17}$$

Hence,

$$|\Omega(\mathbf{x}_p, \mathbf{x}_q)| \le \eta^2 \|\boldsymbol{\delta}(\mathbf{x}_p)\|_2 \|\boldsymbol{\delta}(\mathbf{x}_q)\|_2 \|\mathbf{h}(\mathbf{x}_p)\|_2 \|\mathbf{h}(\mathbf{x}_q)\|_2 \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q). \tag{18}$$

In particular, if the two samples have concentrated memberships on different latent prototypes, then the quantity

$$\sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q)$$

is small, and therefore the overlap between their updates is small.

Proof. From (14),

$$\Delta W_k(\mathbf{x}_p) = -\eta\, g_k(\mathbf{x}_p)\, \boldsymbol{\delta}(\mathbf{x}_p)\, \mathbf{h}(\mathbf{x}_p)^\top$$

and

$$\Delta W_k(\mathbf{x}_q) = -\eta\, g_k(\mathbf{x}_q)\, \boldsymbol{\delta}(\mathbf{x}_q)\, \mathbf{h}(\mathbf{x}_q)^\top.$$

Therefore,

$$\langle \Delta W_k(\mathbf{x}_p), \Delta W_k(\mathbf{x}_q) \rangle_F = \eta^2 g_k(\mathbf{x}_p) g_k(\mathbf{x}_q) \left\langle \boldsymbol{\delta}(\mathbf{x}_p)\mathbf{h}(\mathbf{x}_p)^\top,\, \boldsymbol{\delta}(\mathbf{x}_q)\mathbf{h}(\mathbf{x}_q)^\top \right\rangle_F.$$

Applying Lemma 6.1,

$$\left\langle \boldsymbol{\delta}(\mathbf{x}_p)\mathbf{h}(\mathbf{x}_p)^\top,\, \boldsymbol{\delta}(\mathbf{x}_q)\mathbf{h}(\mathbf{x}_q)^\top \right\rangle_F = \Big(\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)\Big)\Big(\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)\Big).$$

Summing over k gives

$$\Omega(\mathbf{x}_p, \mathbf{x}_q) = \eta^2 \Big(\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)\Big)\Big(\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)\Big) \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q),$$

which proves (17).

The bound (18) follows directly from Cauchy–Schwarz:

$$|\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)| \le \|\boldsymbol{\delta}(\mathbf{x}_p)\|_2 \|\boldsymbol{\delta}(\mathbf{x}_q)\|_2,$$

$$|\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)| \le \|\mathbf{h}(\mathbf{x}_p)\|_2 \|\mathbf{h}(\mathbf{x}_q)\|_2.$$

Substituting these inequalities into (17) yields the result. □

6.4 A Jacobian-based bound for hidden-layer update overlap

The previous proposition isolates the interference-reduction mechanism at the prototype-conditioned output layer. We now derive a related bound for the hidden-layer update. The purpose is not to prove global convergence of the full nonlinear optimizer, but to show that the hidden-layer gradient itself admits a clean Jacobian-based representation, from which a local overlap bound follows.

Let

$$u(x) = W_h x + b_h, \qquad h(x) = \tanh(u(x)),$$

and define the one-sample implemented loss

$$\ell(x, y) = L_{\text{impl}}(x, y) = L_{\text{out}}(x, y) + \lambda L_{\text{loc}}(x),$$

where

$$L_{\text{loc}}(x) = \frac{1}{2}\|h(x) - t(x)\|_2^2.$$

Let

$$\delta_{\text{out}}(x, y) := \nabla_z L_{\text{out}}(z(x), y) \in \mathbb{R}^C,$$

and denote the Jacobians

$$J_z(h) := \frac{\partial z}{\partial h}(h) \in \mathbb{R}^{C\times H}, \qquad J_t(h) := \frac{\partial t}{\partial h}(h) \in \mathbb{R}^{H\times H}.$$

Since the model is trained end-to-end through the full computational graph, the hidden-layer gradient is obtained by the exact chain rule.

Local linearization viewpoint. For a small perturbation Δh around a reference hidden state h, the output map and auxiliary target admit the first-order expansions

$$z(h + \Delta h) = z(h) + J_z(h)\Delta h + o(\|\Delta h\|_2),$$
$$t(h + \Delta h) = t(h) + J_t(h)\Delta h + o(\|\Delta h\|_2).$$

Thus the hidden-space sensitivity of the loss is governed locally by $J_z(h)$ and $J_t(h)$.

Proposition 6.3 (Exact hidden-layer gradient identity)**.** *For the one-hidden-layer geometry-aware model,*

$$\nabla_h \ell(x, y) = J_z(h(x))^\top \delta_{\text{out}}(x, y) + \lambda\big(I - J_t(h(x))\big)^\top \big(h(x) - t(x)\big).$$

Consequently,

$$\nabla_{W_h} \ell(x, y) = \psi(x, y)x^\top,$$

where

$$\psi(x, y) = D(x)\,\gamma(x, y), \qquad D(x) := \operatorname{diag}\big(1 - h_1(x)^2, \ldots, 1 - h_H(x)^2\big),$$

and

$$\gamma(x, y) = J_z(h(x))^\top \delta_{\text{out}}(x, y) + \lambda\big(I - J_t(h(x))\big)^\top \big(h(x) - t(x)\big).$$

Proof. By definition,

$$\ell(x, y) = L_{\text{out}}(z(x), y) + \frac{\lambda}{2}\|h(x) - t(x)\|_2^2.$$

Applying the chain rule to the first term gives

$$\nabla_h L_{\text{out}}(z(x), y) = J_z(h(x))^\top \delta_{\text{out}}(x, y).$$

For the local term, let

$$r(h) = h - t(h).$$

Then

$$L_{\text{loc}}(x) = \frac{1}{2}\|r(h(x))\|_2^2,$$

so

$$\nabla_h L_{\text{loc}}(x) = J_r(h(x))^\top r(h(x)), \qquad J_r(h) = I - J_t(h).$$

Hence

$$\nabla_h \ell(x, y) = J_z(h(x))^\top \delta_{\text{out}}(x, y) + \lambda\big(I - J_t(h(x))\big)^\top \big(h(x) - t(x)\big).$$

Finally, since $h(x) = \tanh(u(x))$ with $u(x) = W_h x + b_h$,

$$\frac{\partial h}{\partial u}(x) = D(x),$$

and therefore

$$\nabla_{W_h} \ell(x, y) = D(x)\, \nabla_h \ell(x, y)\, x^\top = \psi(x, y) x^\top.$$

□

The proposition shows that the hidden-layer update remains rank one at the matrix level, but its left factor is now a nonlinear combination of two terms: a supervised term propagated through $J_z(h)$, and a geometry-aware local term propagated through $I - J_t(h)$.

Lemma 6.4 (Explicit Jacobians)**.** *For the prototype-conditioned model,*

$$J_t(h) = \sum_{k=1}^{K} \phi_k \big(\nabla_h g_k(h)\big)^\top,$$

and

$$J_z(h) = \sum_{k=1}^{K} g_k(h)\, W_k^{\text{exp}} + \sum_{k=1}^{K} z_k^{\text{exp}}(h) \big(\nabla_h g_k(h)\big)^\top.$$

Proof. Since

$$t(h) = \sum_{k=1}^{K} g_k(h) \phi_k,$$

differentiating term by term yields

$$J_t(h) = \sum_{k=1}^{K} \phi_k \big(\nabla_h g_k(h)\big)^\top.$$

Similarly, from

$$z(h) = \sum_{k=1}^{K} g_k(h) z_k^{\exp}(h), \qquad z_k^{\exp}(h) = W_k^{\exp} h + b_k^{\exp},$$

the product rule gives

$$J_z(h) = \sum_{k=1}^{K} \left(g_k(h) \frac{\partial z_k^{\exp}}{\partial h}(h) + z_k^{\exp}(h) \big(\nabla_h g_k(h)\big)^\top \right).$$

Since

$$\frac{\partial z_k^{\exp}}{\partial h}(h) = W_k^{\exp},$$

the result follows. □

We now quantify the overlap between two hidden-layer updates.

Proposition 6.5 (Jacobian-based hidden-layer overlap formula)**.** *For two samples (x_p, y_p) and (x_q, y_q), define*

$$\Delta W_h(x, y) := -\eta \nabla_{W_h} \ell(x, y) = -\eta\, \psi(x, y) x^\top.$$

Their hidden-layer update overlap is

$$\Omega_h(x_p, x_q) := \langle \Delta W_h(x_p, y_p), \Delta W_h(x_q, y_q) \rangle_F .$$

Then

$$\Omega_h(x_p, x_q) = \eta^2 \big(\psi(x_p, y_p)^\top \psi(x_q, y_q)\big)\big(x_p^\top x_q\big).$$

Hence,

$$|\Omega_h(x_p, x_q)| \le \eta^2 \|x_p\|_2 \|x_q\|_2 \|\psi(x_p, y_p)\|_2 \|\psi(x_q, y_q)\|_2.$$

Since $\|D(x)\|_2 \le 1$ for tanh*, it follows that*

$$|\Omega_h(x_p, x_q)| \le \eta^2 \|x_p\|_2 \|x_q\|_2 \|\gamma(x_p, y_p)\|_2 \|\gamma(x_q, y_q)\|_2.$$

Proof. From Proposition 6.3,

$$\Delta W_h(x, y) = -\eta\, \psi(x, y) x^\top.$$

Applying the Frobenius rank-one identity from Lemma 6.1 gives

$$\Omega_h(x_p, x_q) = \eta^2 \left\langle \psi(x_p, y_p) x_p^\top,\, \psi(x_q, y_q) x_q^\top \right\rangle_F = \eta^2 \big(\psi(x_p, y_p)^\top \psi(x_q, y_q)\big)\big(x_p^\top x_q\big).$$

The bound then follows by Cauchy–Schwarz. Since each diagonal entry of $D(x)$ is $1 - h_i(x)^2 \in [0, 1]$, we have $\|D(x)\|_2 \le 1$, which yields

$$\|\psi(x, y)\|_2 \le \|\gamma(x, y)\|_2.$$

□

The preceding proposition is exact. To obtain a more interpretable local bound, we now impose bounded-Jacobian assumptions in a neighborhood of hidden space.

Corollary 6.6 (Local Jacobian-norm bound)**.** *Assume that there exists a neighborhood $\mathcal{U} \subset \mathbb{R}^H$ such that for all $h \in \mathcal{U}$,*

$$\|J_z(h)\|_2 \le M_z, \qquad \|J_t(h)\|_2 \le M_t,$$

and for all samples whose hidden states lie in $\mathcal{U}$,

$$\|\delta_{\text{out}}(x,y)\|_2 \le B_{\text{out}}, \qquad \|h(x) - t(x)\|_2 \le B_{\text{loc}}.$$

Then for any two such samples,

$$|\Omega_h(x_p, x_q)| \le \eta^2 \|x_p\|_2 \|x_q\|_2 \Big(M_z B_{\text{out}} + \lambda(1 + M_t) B_{\text{loc}} \Big)^2.$$

Proof. From Proposition 6.3,

$$\|\gamma(x,y)\|_2 \le \|J_z(h(x))^\top \delta_{\text{out}}(x,y)\|_2 + \lambda \left\| \big(I - J_t(h(x))\big)^\top (h(x) - t(x)) \right\|_2.$$

Therefore,

$$\|\gamma(x,y)\|_2 \le \|J_z(h(x))\|_2 \, \|\delta_{\text{out}}(x,y)\|_2 + \lambda \big(1 + \|J_t(h(x))\|_2\big) \|h(x) - t(x)\|_2.$$

Using the assumed bounds yields

$$\|\gamma(x,y)\|_2 \le M_z B_{\text{out}} + \lambda(1 + M_t) B_{\text{loc}}.$$

Substituting this into Proposition 6.5 gives the result. □

The next statement connects the bound more explicitly to prototype concentration. It shows that, away from prototype boundaries, the hidden-layer signal is close to a region-specific backpropagated term plus a region-specific auxiliary correction.

Corollary 6.7 (Dominant-prototype approximation)**.** *Assume that for a sample x there exists an index $k^\star(x)$ such that*

$$g_{k^\star(x)}(x) \ge 1 - \varepsilon, \qquad \sum_{k \ne k^\star(x)} g_k(x) \le \varepsilon,$$

for some $\varepsilon \in [0,1]$. Assume further that

$$\|W_k^{\text{exp}}\|_2 \le M_W, \qquad \|z_k^{\text{exp}}(h)\|_2 \le B_{\text{exp}}, \qquad \|\phi_k\|_2 \le M_\phi,$$

for all k, and that

$$\|\nabla_h g_k(h)\|_2 \le \rho$$

for all k in a neighborhood of interest. Then

$$\|J_z(h) - W^{\exp}_{k^\star(x)}\|_2 \le 2\varepsilon M_W + K\rho B_{\exp},$$

$$\|J_t(h)\|_2 \le K\rho M_\phi, \qquad \|t(h) - \phi_{k^\star(x)}\|_2 \le 2\varepsilon M_\phi.$$

Consequently,

$$\Big\|\gamma(x,y) - \Big([W^{\exp}_{k^\star(x)}]^\top \delta_{\text{out}}(x,y) + \lambda\big(h(x) - \phi_{k^\star(x)}\big)\Big)\Big\|_2 \le R(x,y),$$

where

$$R(x,y) = \big(2\varepsilon M_W + K\rho B_{\exp}\big)\|\delta_{\text{out}}(x,y)\|_2 + \lambda\Big(2\varepsilon M_\phi + K\rho M_\phi \|h(x) - t(x)\|_2\Big).$$

Proof. Using Lemma 6.4,

$$J_z(h) - W^{\exp}_{k^\star} = \sum_{k=1}^K g_k(h) W^{\exp}_k - W^{\exp}_{k^\star} + \sum_{k=1}^K z^{\exp}_k(h)(\nabla_h g_k(h))^\top.$$

For the first term,

$$\left\|\sum_{k=1}^K g_k(h) W^{\exp}_k - W^{\exp}_{k^\star}\right\|_2 \le |g_{k^\star}(h) - 1|\, \|W^{\exp}_{k^\star}\|_2 + \sum_{k \ne k^\star} g_k(h) \|W^{\exp}_k\|_2 \le 2\varepsilon M_W.$$

For the second term,

$$\left\|\sum_{k=1}^K z^{\exp}_k(h)(\nabla_h g_k(h))^\top\right\|_2 \le \sum_{k=1}^K \|z^{\exp}_k(h)\|_2\, \|\nabla_h g_k(h)\|_2 \le K\rho B_{\exp}.$$

Hence

$$\|J_z(h) - W^{\exp}_{k^\star}\|_2 \le 2\varepsilon M_W + K\rho B_{\exp}.$$

Similarly,

$$\|J_t(h)\|_2 = \left\|\sum_{k=1}^K \phi_k (\nabla_h g_k(h))^\top\right\|_2 \le \sum_{k=1}^K \|\phi_k\|_2\, \|\nabla_h g_k(h)\|_2 \le K\rho M_\phi.$$

Also,

$$t(h) - \phi_{k^\star} = (g_{k^\star}(h) - 1)\phi_{k^\star} + \sum_{k \ne k^\star} g_k(h)\phi_k,$$

so

$$\|t(h) - \phi_{k^\star}\|_2 \le |g_{k^\star}(h) - 1|\, \|\phi_{k^\star}\|_2 + \sum_{k \ne k^\star} g_k(h) \|\phi_k\|_2 \le 2\varepsilon M_\phi.$$

Finally,

$$\gamma - \Big(W^{\exp\,\top}_{k^\star} \delta_{\text{out}} + \lambda(h - \phi_{k^\star})\Big) = \big(J_z^\top - W^{\exp\,\top}_{k^\star}\big)\delta_{\text{out}} + \lambda\Big((h-t) - J_t^\top(h-t) - (h - \phi_{k^\star})\Big).$$

The second bracket equals

$$(\phi_{k^\star} - t) - J_t^\top (h - t).$$

Applying the triangle inequality together with the bounds above yields the stated residual bound. □

Corollary 6.7 formalizes the following intuition. When a sample lies well inside a latent region, the membership map is concentrated and locally smooth, so the hidden-layer gradient is well approximated by a region-specific back-propagated signal $[W^{\text{exp}}_{k^\star(x)}]^\top \delta_{\text{out}}(x, y)$ plus a region-specific auxiliary correction $\lambda(h(x) - \phi_{k^\star(x)})$. The approximation degrades only near prototype boundaries or when the memberships are highly diffuse.

Remark on scope. This subsection still does not constitute a full convergence analysis of the exact nonlinear optimizer. Rather, it provides a local Jacobian-based control of the hidden-layer update and shows explicitly how prototype concentration and membership smoothness influence the hidden credit signal.

6.5 Interpretation

Proposition 6.2 shows that the overlap between two sample-induced updates is scaled by the membership similarity factor

$$S(\mathbf{x}_p, \mathbf{x}_q) = \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q). \tag{19}$$

This factor lies in $[0, 1]$. It is large when the two samples activate similar latent regions and small when they activate different latent regions. Therefore, geometry-aware gating naturally reduces coupling between samples that belong to different parts of hidden representation space.

By contrast, in a globally shared output layer without prototype-conditioned gating, there is no explicit factor analogous to (19) that suppresses cross-region interaction. The proposition therefore formalizes the key qualitative mechanism behind the proposed model.

6.6 A useful corollary

Corollary 6.8. *Assume that for sample $\mathbf{x}_p$ there exists an index k_p such that $g_{k_p}(\mathbf{x}_p) \geq 1 - \varepsilon$, and for sample $\mathbf{x}_q$ there exists an index $k_q \neq k_p$ such that $g_{k_q}(\mathbf{x}_q) \geq 1 - \varepsilon$, where $\varepsilon \in [0, 1]$. Then*

$$\sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q) \leq 2\varepsilon - \varepsilon^2. \tag{20}$$

Consequently,

$$|\Omega(\mathbf{x}_p, \mathbf{x}_q)| \leq \eta^2 \|\boldsymbol{\delta}(\mathbf{x}_p)\|_2 \|\boldsymbol{\delta}(\mathbf{x}_q)\|_2 \|\mathbf{h}(\mathbf{x}_p)\|_2 \|\mathbf{h}(\mathbf{x}_q)\|_2 (2\varepsilon - \varepsilon^2). \tag{21}$$

Proof. Since $g_{k_p}(\mathbf{x}_p) \geq 1 - \varepsilon$, the total mass on all other indices is at most ε. Likewise, since $g_{k_q}(\mathbf{x}_q) \geq 1 - \varepsilon$ and $k_q \neq k_p$, the mass outside k_q is at most ε. Therefore, the dot product between the two membership vectors is maximized when the remaining mass is concentrated as much as possible, which yields

$$\sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q) \leq \varepsilon + \varepsilon(1 - \varepsilon) = 2\varepsilon - \varepsilon^2.$$

Substituting this into (18) completes the proof. □

6.7 Remark on scope

The above result does not establish global convergence of the full nonlinear model trained by (12), nor does it claim that all forms of interference are eliminated. Rather, it isolates one mathematically precise mechanism: when geometry-aware memberships concentrate on different latent regions, the overlap between the corresponding gated update components is explicitly suppressed. This provides a concrete theoretical basis for the claim that local geometry can reduce cross-region interference.

7 Results

We evaluate whether geometry-aware local conditioning improves over a standard backpropagation (BP) baseline while preserving the same end-to-end training pipeline. The results are organized around three questions: (i) does the proposed method improve predictive performance, (ii) does it alter optimization dynamics in a way consistent with the mechanism analysis, and (iii) do the learned prototypes capture meaningful structure in hidden space.

7.1 Experimental protocol

Across all experiments, the standard BP baseline and the proposed geometry-local model share the same backbone, optimizer, training budget, and random seeds; the only change is the addition of trainable hidden-space prototypes, prototype-conditioned output experts, and the auxiliary local loss. We report mean ± 95% confidence interval over five seeds.

For the dummy task and MNIST, we use a one-hidden-layer MLP with hidden dimension 16 trained for 80 epochs. The geometry-local model uses $\lambda_{\text{local}} = 0.02$, $K = 8$ prototypes, and temperature $\tau = 3.0$. For Fashion-MNIST, we use the same MLP with a 30-epoch budget and perform a grid search over $\lambda_{\text{local}} \in \{0, 0.01, 0.02, 0.05\}$, $K \in \{4, 8, 12\}$, and $\tau \in \{0.5, 1.0, 3.0\}$; the reported geometry-local result is the validation-selected configuration. For CIFAR-10, we use a small CNN backbone followed by a hidden representation of dimension 256 and train for 50 epochs; the geometry-local model uses $\lambda_{\text{local}} = 0.01$, $K = 8$, and $\tau = 1.0$.

Table 1: Main quantitative results over five seeds. Lower is better for test loss and epochs-to-target, while higher is better for test accuracy. The Fashion-MNIST geometry-local row corresponds to the validation-selected configuration.

Dataset	Method	Test accuracy ↑	Test loss ↓	Epochs-to-target ↓
Dummy	Standard BP	0.9975 ± 0.0022	0.0066 ± 0.0029	2.4 ± 0.68
Dummy	Geometry-local	$\mathbf{0.9988 \pm 0.0023}$	$\mathbf{0.0059 \pm 0.0061}$	$\mathbf{1.0 \pm 0.0}$
MNIST	Standard BP	0.9036 ± 0.0034	0.4599 ± 0.0315	–
MNIST	Geometry-local	$\mathbf{0.9108 \pm 0.0040}$	$\mathbf{0.4555 \pm 0.0136}$	–
Fashion-MNIST	Standard BP	$\mathbf{0.8399 \pm 0.0040}$	$\mathbf{0.4598 \pm 0.0065}$	–
Fashion-MNIST	Geometry-local	0.8396 ± 0.0041	0.5030 ± 0.0168	–
CIFAR-10	Standard BP	0.7800 ± 0.0131	0.9536 ± 0.0924	$\mathbf{6.2 \pm 1.04}$
CIFAR-10	Geometry-local	$\mathbf{0.7896 \pm 0.0164}$	$\mathbf{0.8430 \pm 0.0783}$	7.8 ± 1.62

In addition to final test accuracy and test loss, we also track two optimization diagnostics: the within-epoch standard deviation of batch losses and the within-epoch standard deviation of hidden-layer gradient norms. These diagnostics are useful because the theory does not predict uniformly smaller gradients; rather, it predicts more localized, geometry-dependent credit assignment.

7.2 Main quantitative comparison

The proposed method yields its clearest gain on the synthetic dummy task. It reaches the target validation accuracy in 1.0 ± 0.0 epochs, compared to 2.4 ± 0.68 epochs for standard BP, while also slightly improving final test accuracy and test loss. Since this task is explicitly multimodal, this result is consistent with the central hypothesis of the paper: when different regions of the data manifold would otherwise share a single global update, geometry-aware local conditioning can reduce destructive interference and speed up learning.

On MNIST, the geometry-local model improves both test accuracy and test loss relative to standard BP. The gain in accuracy is modest in absolute terms, from 0.9036 ± 0.0034 to 0.9108 ± 0.0040, but it is consistent across all five seeds. Importantly, the method also slightly reduces within-epoch batch-loss variability. This suggests that the added geometric structure does not merely perturb training, but can produce a cleaner optimization trajectory while improving final generalization.

Fashion-MNIST is the main boundary case. Under validation-based model selection, the best geometry-local configuration achieves a higher best validation accuracy than standard BP (0.8584 ± 0.0098 versus 0.8542 ± 0.0065), but this does not translate into a test improvement for the selected configuration. The final test accuracy is essentially tied, while the geometry-local test loss is higher. This is an important negative-to-mixed result: it suggests that the method is not uniformly beneficial, and that its effect depends on how well the prototype geometry and local regularization strength are matched to the task.

At the same time, the Fashion-MNIST grid search also shows that the

method remains competitive under nearby hyperparameter choices. In particular, some non-selected configurations slightly exceed the BP baseline in test accuracy, e.g. $(\lambda_{\text{local}}, K, \tau) = (0.02, 12, 3.0)$ reaches 0.8422 ± 0.0048. We therefore interpret Fashion-MNIST not as evidence against the method, but as evidence of sensitivity: the geometry-aware inductive bias is plausible, but its benefit is more tuning-dependent on this dataset.

On CIFAR-10, the geometry-local model again improves the final solution. It increases test accuracy from 0.7800 ± 0.0131 to 0.7896 ± 0.0164 and reduces test loss from 0.9536 ± 0.0924 to 0.8430 ± 0.0783. Moreover, the geometry-local model attains lower test loss in all five seeds and higher test accuracy in four of the five seeds. This is the strongest evidence that the proposed mechanism remains useful beyond the shallow MLP setting and continues to help when the hidden representation is produced by a CNN backbone.

The CIFAR-10 result is also informative in a second sense. Although the geometry-local model converges to a better final solution, it does *not* reach the target validation threshold faster; standard BP reaches the threshold in 6.2 ± 1.04 epochs, whereas the geometry-local model requires 7.8 ± 1.62 epochs. This suggests that the proposed method should not be interpreted as a universal accelerator of early optimization. Its main benefit is instead a better final representation and a better final predictor, especially in more heterogeneous settings.

7.3 Optimization dynamics

From Figs. 2, 3, a recurring pattern across datasets is that the geometry-local model yields comparable or slightly lower within-epoch batch-loss variability, while simultaneously producing larger hidden-gradient variability. This pattern is visible on MNIST and CIFAR-10, and to a lesser extent on the dummy task once the early transient phase has passed.

This behavior is consistent with the mechanism analysis developed earlier in the paper. The goal of the geometry-aware design is not to globally suppress gradients. Rather, it is to make the effective update more sample-dependent by routing each hidden representation through soft prototype memberships and prototype-conditioned experts. Under this view, larger hidden-gradient variability is not a failure mode; it is evidence that the credit signal becomes more differentiated across regions of hidden space, instead of being homogenized by a single global output map.

The loss-variability curves support the same interpretation. On MNIST and CIFAR-10, the geometry-local model achieves either similar or slightly lower within-epoch batch-loss dispersion despite the more structured hidden-layer dynamics. This suggests that local geometric modulation increases selectivity without destabilizing optimization.

7.4 Representation geometry

From Figs. 4, 5, the hidden-space visualizations provide qualitative support for the proposed mechanism.

On the dummy task, the learned prototypes spread across the occupied hidden regions rather than collapsing to a single point. Since the data are strongly multimodal, this is exactly the regime in which prototype-conditioned experts should help: different regions can be assigned different effective output mappings while still sharing the same backbone representation.

On MNIST, the hidden representations occupy a bounded region shaped by the tanh hidden layer, and the learned prototypes are distributed across that bounded manifold. The key observation is again non-collapse: the model does not learn a single global template, but multiple local anchors that can be reused across samples.

On CIFAR-10, the PCA projection of the hidden representation shows that the learned prototypes occupy distinct regions of the feature space produced by the CNN backbone. Even though the projection is only qualitative, it supports the claim that the method is using meaningful geometric structure in feature space rather than learning a degenerate gating pattern.

7.5 Overall findings

Taken together, the experiments provide meaningful but appropriately scoped support for the theory and theme of the paper. The strongest support appears in the dummy task, where the multimodal structure is explicit and the geometry-local model reaches the target much faster. MNIST shows a smaller but consistent gain in final performance. CIFAR-10 shows that the same idea remains beneficial in a CNN setting, where the final test accuracy and test loss are both improved. Fashion-MNIST is mixed, which is scientifically useful: it shows that the method is not automatically superior under all configurations and that the quality of the learned local geometry matters.

The main empirical takeaway is therefore not that geometry-local learning uniformly dominates standard BP. Rather, the evidence suggests a more precise claim: incorporating local geometry into the prediction pathway and into the hidden-space regularization can improve final performance and produce more differentiated internal credit assignment, especially when the task contains heterogeneous or multimodal structure. This is exactly the regime targeted by the mechanism analysis, and the observed optimization and representation trends are consistent with that interpretation.

```
================ All geometry-local configurations (sorted) ================

lambda=0.01 | protos=8  | tau=1.0 | best_val_acc=0.8584 | test_acc=0.8396 | test_loss=0.5030
lambda=0.0  | protos=8  | tau=1.0 | best_val_acc=0.8582 | test_acc=0.8404 | test_loss=0.4802
lambda=0.02 | protos=8  | tau=3.0 | best_val_acc=0.8576 | test_acc=0.8400 | test_loss=0.4955
lambda=0.02 | protos=12 | tau=1.0 | best_val_acc=0.8574 | test_acc=0.8367 | test_loss=0.5243
lambda=0.01 | protos=8  | tau=3.0 | best_val_acc=0.8570 | test_acc=0.8417 | test_loss=0.4795
lambda=0.05 | protos=8  | tau=3.0 | best_val_acc=0.8564 | test_acc=0.8375 | test_loss=0.5213
lambda=0.05 | protos=12 | tau=3.0 | best_val_acc=0.8558 | test_acc=0.8391 | test_loss=0.5046
lambda=0.02 | protos=12 | tau=3.0 | best_val_acc=0.8556 | test_acc=0.8422 | test_loss=0.4856
lambda=0.01 | protos=12 | tau=1.0 | best_val_acc=0.8554 | test_acc=0.8407 | test_loss=0.5002
lambda=0.0  | protos=12 | tau=3.0 | best_val_acc=0.8552 | test_acc=0.8410 | test_loss=0.4685
lambda=0.01 | protos=8  | tau=0.5 | best_val_acc=0.8548 | test_acc=0.8387 | test_loss=0.5149
lambda=0.02 | protos=4  | tau=3.0 | best_val_acc=0.8548 | test_acc=0.8382 | test_loss=0.5179
lambda=0.0  | protos=8  | tau=3.0 | best_val_acc=0.8546 | test_acc=0.8413 | test_loss=0.4650
lambda=0.02 | protos=8  | tau=1.0 | best_val_acc=0.8544 | test_acc=0.8409 | test_loss=0.5156
lambda=0.01 | protos=12 | tau=3.0 | best_val_acc=0.8544 | test_acc=0.8401 | test_loss=0.4805
lambda=0.02 | protos=4  | tau=1.0 | best_val_acc=0.8544 | test_acc=0.8371 | test_loss=0.5204
lambda=0.0  | protos=12 | tau=0.5 | best_val_acc=0.8538 | test_acc=0.8374 | test_loss=0.4982
lambda=0.0  | protos=8  | tau=0.5 | best_val_acc=0.8536 | test_acc=0.8360 | test_loss=0.4927
lambda=0.0  | protos=12 | tau=1.0 | best_val_acc=0.8532 | test_acc=0.8379 | test_loss=0.4894
lambda=0.05 | protos=4  | tau=3.0 | best_val_acc=0.8530 | test_acc=0.8341 | test_loss=0.5518
lambda=0.02 | protos=12 | tau=0.5 | best_val_acc=0.8528 | test_acc=0.8360 | test_loss=0.5334
lambda=0.05 | protos=12 | tau=1.0 | best_val_acc=0.8526 | test_acc=0.8384 | test_loss=0.5329
lambda=0.0  | protos=4  | tau=1.0 | best_val_acc=0.8526 | test_acc=0.8378 | test_loss=0.4760
lambda=0.05 | protos=8  | tau=1.0 | best_val_acc=0.8526 | test_acc=0.8360 | test_loss=0.5482
lambda=0.01 | protos=4  | tau=1.0 | best_val_acc=0.8522 | test_acc=0.8358 | test_loss=0.5026
lambda=0.02 | protos=4  | tau=0.5 | best_val_acc=0.8520 | test_acc=0.8362 | test_loss=0.5322
lambda=0.01 | protos=4  | tau=3.0 | best_val_acc=0.8520 | test_acc=0.8357 | test_loss=0.4964
lambda=0.0  | protos=4  | tau=0.5 | best_val_acc=0.8518 | test_acc=0.8367 | test_loss=0.4787
lambda=0.01 | protos=4  | tau=0.5 | best_val_acc=0.8516 | test_acc=0.8369 | test_loss=0.5068
lambda=0.05 | protos=12 | tau=0.5 | best_val_acc=0.8516 | test_acc=0.8350 | test_loss=0.5535
lambda=0.02 | protos=8  | tau=0.5 | best_val_acc=0.8514 | test_acc=0.8387 | test_loss=0.5322
lambda=0.05 | protos=8  | tau=0.5 | best_val_acc=0.8512 | test_acc=0.8350 | test_loss=0.5674
lambda=0.05 | protos=4  | tau=1.0 | best_val_acc=0.8494 | test_acc=0.8345 | test_loss=0.5665
lambda=0.0  | protos=4  | tau=3.0 | best_val_acc=0.8490 | test_acc=0.8370 | test_loss=0.4650
lambda=0.01 | protos=12 | tau=0.5 | best_val_acc=0.8488 | test_acc=0.8378 | test_loss=0.5208
lambda=0.05 | protos=4  | tau=0.5 | best_val_acc=0.8480 | test_acc=0.8302 | test_loss=0.5816
```

Table 2: Grid search for Fashion MNIST dataset

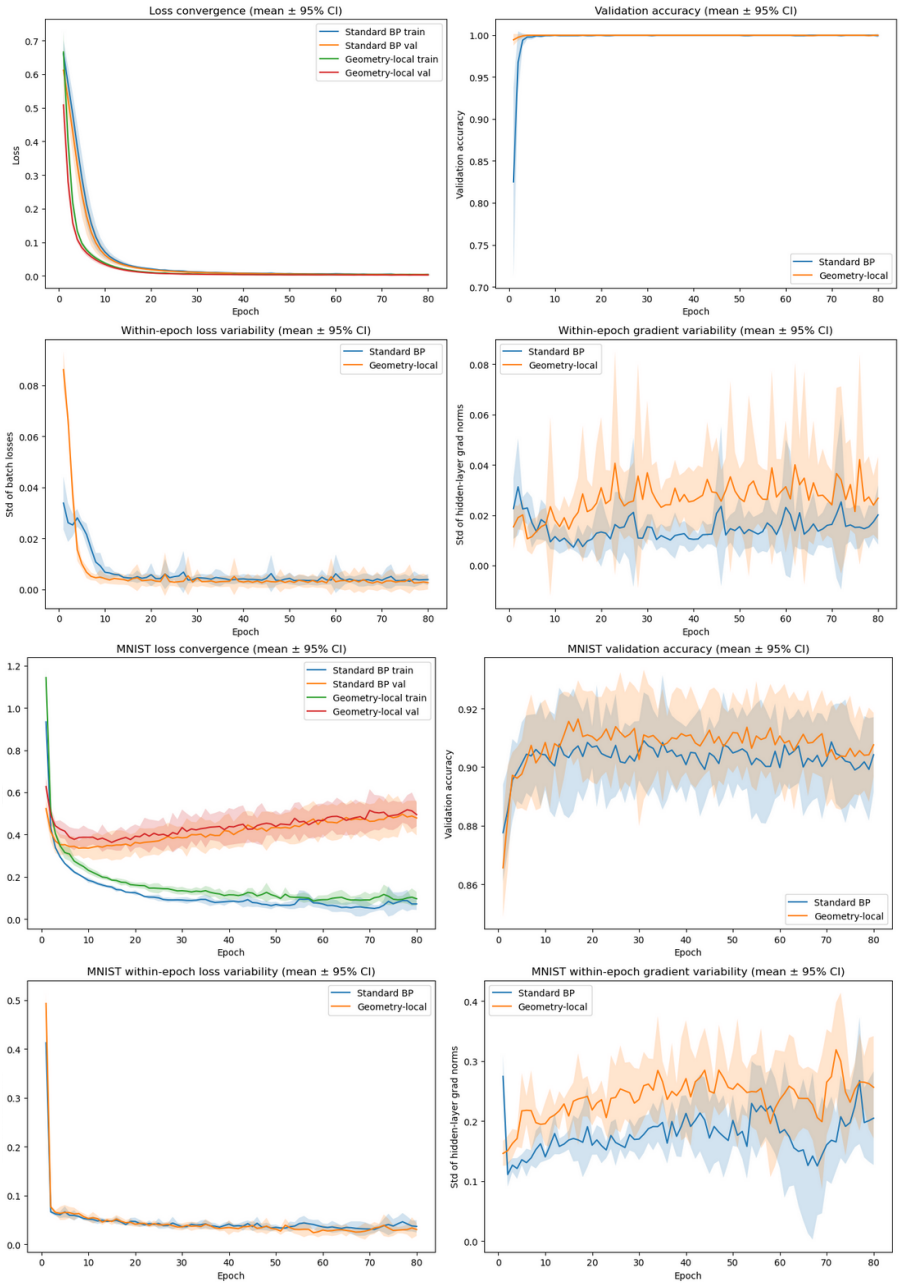

Figure 2: Results for dummy toy example, MNIST dataset

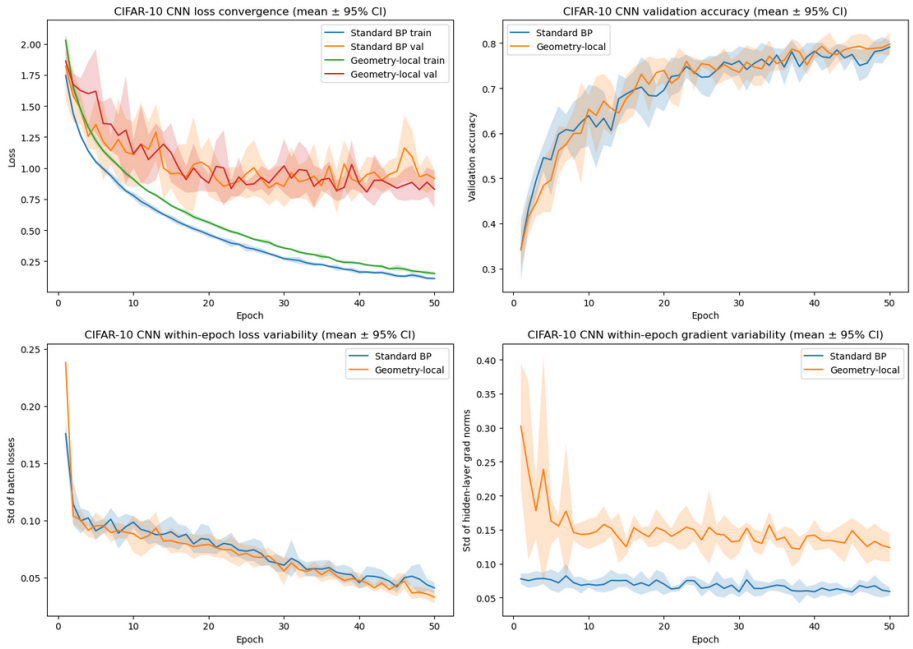

Figure 3: Results for CIFAR10 dataset

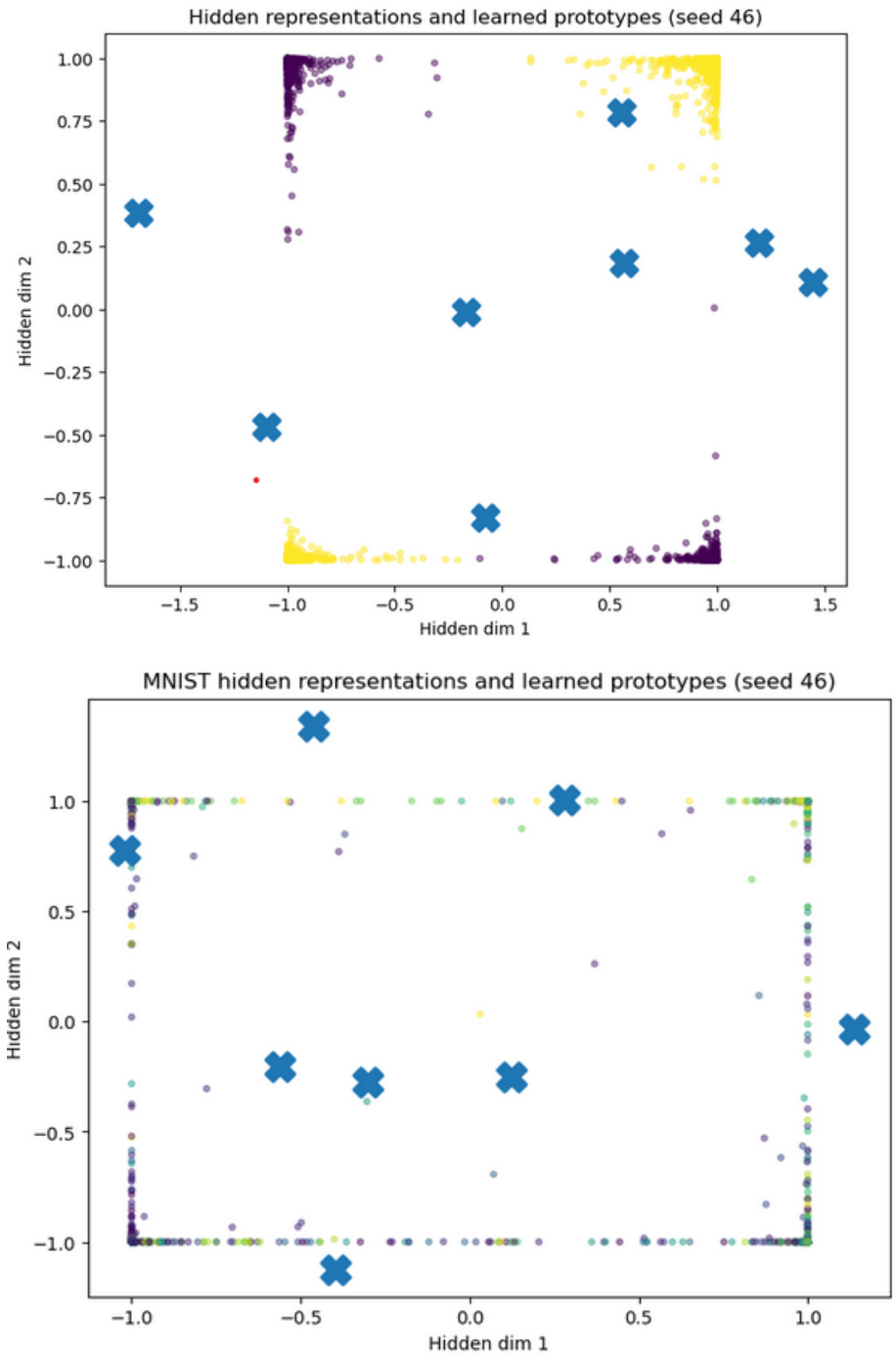

Figure 4: Hidden representation for dummy toy example, MNIST dataset

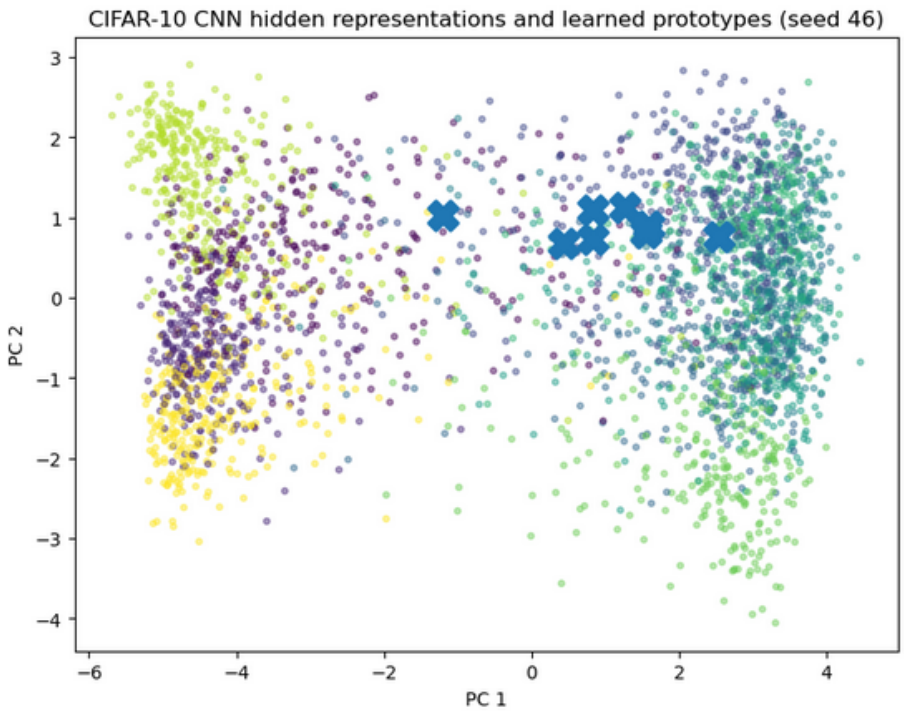

Figure 5: Hidden representation for CIFAR10 dataset